1. You are developing a high-performance visual simulation that tracks **millions of particles**. Each particle has unique properties (position, velocity, mass, charge, etc. The core operation is not looking up individual particles by ID. It's applying the **same physics function** (e.g., compute forces, update position) to **all particles** in parallel on a GPU.
   a) Based on the above requirement recommend suitable data structure and why it is your choice that would be suitable to store this information.
   b) Also, recommend a data structure that would not be suitable for this with proper justification.

2. An e-commerce platform operates globally, selling millions of products across dozens of regions. They need to track:

   - Real-time stock levels for each product in each warehouse
   - Price variations by region and currency
   - Product metadata (name, category, brand)

Problem with Simple Keys

Using simple concatenated keys like "product_12345:warehouse_nyc:stock" creates chaos:

a) To find all warehouses with stock for product 12345 → must query millions of keys with prefix scanning. product_12345:*:stock (scans millions)

b) To adjust all prices in Europe by 10%("product_12345_price_USD, product_12345_price_EUR, product_12345_price_GBP etc. **Scan EVERY key in the entire database** (millions of keys) **For each key:**
   i. Check if it contains _price_ (to identify price keys)
   ii. Check if it ends with _EUR or _GBP or other European currencies

c) To get all products in "electronics" category
product_12345_name: "Smartphone X"
product_12345_price: "299.99"
product_12345_stock: "150"
product_67890_name: "T-Shirt"
product_67890_price: "19.99"
product_67890_stock: "500"
product_55555_name: "Laptop Pro"
product_55555_price: "1299.99"
product_55555_stock: "75"
product_99999_name: "Coffee Mug"
product_99999_price: "12.99"
product_99999_stock: "1000"
product_12345_category: "electronics"
product_67890_category: "clothing"
product_55555_category: "electronics"
product_99999_category: "home"

a) Recommend a solution for search to be performed based on category? Mention the advantage and disadvantage of the same.

b) Recommend a key so that these issues can be handled without creating index and perform less scans. The format of the key is given

Format: REGION:CATEGORY:ID:ATTRIBUTE:SUB_ATTRIBUTE

For each case (stock levels in each warehouse, price in Different Currencies/Regions

, product metadata) recommend suitable elements in the above format repressing countrywide category with the product id with the attribute and sub-attribute for stock, prices in different currencies (USD,EUR,GBP,JPY), and product metadata like having name as smartphone, brand as techCorp, weight as 0.5 kg)

3. Consider a social media platform storing serialized user session objects (containing preferences, auth tokens, cached feed data) as values in a key-value store. As user sessions grow more complex over time, individual values balloon from 2KB to 50KB each.
   A social media platform starts with simple session objects (2KB):

```
{
  "user_id": "12345",
  "auth_token": "abc123",
  "theme": "dark",
  "last_login": "2024-01-15"
}
```

Over time, sessions evolve to include (50KB):

```
{
  "user_id": "12345",
  "auth_token": "abc123",
  "preferences": {...},  // 5KB
  "cached_feed": [...],  // 40KB of post IDs and metadata
  "notifications": [...], // 3KB
  "recent_searches": [...], // 2KB
  "draft_posts": {...}   // Additional complexity
}
```

This Undermines Key-Value Database Features
1. Speed Degradation
2. Scalability Constraints
Based on the above given information fill the table below

Chunk Type	**Size**	**Access Frequency (High,VH,L,M)**	**Eviction Policy (Never,LRU,Manual)**	**Storage Tier (Memory/SSD/Cache)**
Auth & Core	2KB			
Preferences	5KB			
Recent Feed	16KB			

Chunk Type	Size	Access Frequency (High,VH,L,M)	Eviction Policy (Never,LRU,Manual)	Storage Tier (Memory/SSD/Cache)
Older Feed	16KB			
Notifications	3KB			
Archive Data	8KB			